ADF GEN AI HACKATHON 2026

AE Rule Engine

Configuration Management System

Applied Data Finance · FinTelligence AI

1. Team Details

Team Name	**Fintelligence AI**
Team Members	**Aakash V, Balaji S, Gurram Varalakshmi, Kanisha R, Poornamala S**
Team Lead Name	**BALAJI S**
Team Lead Email	**balajis@applieddatafinance.com**
Team Lead Phone	**+91 9003090545**
Business Unit	**Core Product Engineering**
Submission Date	**13th April 2026**

2. Problem Statement

Problem Title

Manual, Error-Prone Affiliate Engine Rule Management with No Audit Trail or Approval Governance

Business Function / Area Impacted

Affiliate Engine (AE) - Underwriting Decision Logic, Credit Risk Operations, Loan Originations Technology

Problem Summary

The Affiliate Engine (AE) is the core decision-making system at Applied Data Finance that governs whether a customer loan application from an affiliate partner (Quin Street, Lending Tree, Money Lion, Monevo, Credit Karma Pre-Qualify, Credit Match Pre-Qualify, Credit Match ACT) results in an offer, decline, or referral. The engine evaluates applicants against a sophisticated matrix of rules covering credit scores, income ratios, fraud indicators, state eligibility, channel-specific cutoffs, and internal model scores.

Today, every change to these rules — adjusting a cutoff from 500 to 600, toggling a state on or off, updating a segment condition — is managed through manual Excel spreadsheets maintained by the Underwriting Analytics team. The Underwriting team emails the specifications, and engineering manually transposes them into code or configuration, with no systematic validation, no diff tracking, no approval gate, and no rollback mechanism. This creates a high-risk, low-visibility process for one of the most business-critical systems at ADF.

Current Process Today

1. UW Analytics team updates the AE rule specification in Excel (AE_Sample_Spec.xlsx), highlighting changed cells in yellow
2. Updated Excel is emailed to the engineering team or shared via Confluence
3. Developer manually reads the highlighted cells and interprets the intent (cutoff change, new rule, operator change, etc.)
4. Engineer modifies the production configuration file or database record directly
5. A second engineer performs an informal code review — no standardized before/after diff
6. Change is deployed to production with no time-stamped audit record linking the change to the original Excel specification
7. If an issue arises, root-cause requires combing through email chains and git history to reconstruct what changed and why

Key Pain Points

- No single source of truth — Excel, email, and code are out of sync

- No formal approval workflow — changes can go to production without a manager sign-off
- No audit trail — who changed what cutoff, when, and based on which business instruction is untracked
- High risk of transcription errors — manually copying cutoff values from Excel into config
- No environment separation — TEST and PROD configs drift silently
- No version history — impossible to see what rules looked like at a prior point in time
- UW spec and engine config alignment is manual and error-prone — a highlighted yellow cell might be missed
- No automated cross-check between the UW Excel specification and the engine's current configuration

Why This Problem Matters to ADF

- The Affiliate Engine directly controls ADF's loan origination volume and credit risk exposure. An incorrect cutoff — a single number being 500 instead of 600 — can expose ADF to higher default risk or unnecessarily decline creditworthy customers, both of which have direct P&L consequences.
- Regulatory and compliance requirements also demand traceable, auditable change management for credit decision rules. The current manual process is not scalable as ADF grows its affiliate partner network, and it creates unacceptable operational risk for a company that relies on precise, repeatable, data-driven lending decisions.

Who Is Affected by This Problem

- **Underwriting Analytics (UW) Team — authors of rule specifications, currently with no visibility into whether their intent was implemented correctly**
- **Senior Developers and Leads (Engineering) — spend significant time manually interpreting and implementing Excel-sourced changes**
- **Approvers / Risk Managers — cannot perform structured review of rule changes; currently sign off via email**
- **Compliance & Audit — no readily available change log for regulatory examination**
- **Affiliate Partners — affected when rule errors cause incorrect decisioning on their traffic**
- **ADF Leadership — unable to audit or report on rule change history for business reviews**

Problem Background

- Applied Data Finance's Affiliate Engine evaluates loan applications from multiple partner channels against a complex matrix of credit rules, cutoff thresholds, state eligibility constraints, and channel-specific segment definitions.
- Currently, all rule changes are managed through manually annotated Excel spreadsheets emailed between the Underwriting Analytics team and engineering. There is no structured approval workflow, no field-level diff tracking, no version history, and no automated validation that the engine's configuration matches the UW team's intent.

- A single misread cutoff value or missed yellow-highlighted cell can silently alter credit risk decisions across thousands of applications. The absence of a formal review process also creates compliance exposure: regulatory requirements demand documented, auditable evidence of why a credit decision rule changed, on what date, and who authorised it.
- As ADF's affiliate partner network grows — currently including Quin Street, Lending Tree, Money Lion, Monevo, Credit Karma Pre-Qualify, and Credit Match — the volume of configuration changes will increase proportionally. The current ad-hoc, spreadsheet-driven process cannot scale, and the cost of an undetected configuration error in a high-volume lending environment is substantial both financially and reputationally.

3. Proposed Solution

Solution Title

AE Rule Engine — AI-Powered Configuration Management System with UW Excel Ingestion, Embedded Rule Execution, and Flyway-Style Version Control

What the Solution Does

The AE Rule Engine is a full-stack enterprise configuration management platform that transforms ADF's Affiliate Engine rule lifecycle from a manual, Excel-and-mail-driven process into a governed, auditable, AI-assisted workflow. It provides a single source of truth for all AE decision rules, cutoff configurations, state constraints, and channel-specific settings — with structured approval gates, complete version history, and a purpose-built AI ingestion module that reads the UW team's annotated Excel specifications and automatically generates typed change suggestions.

How the Solution Works

Config Management Layer

- The platform stores all 20+ Affiliate Engine rules in a structured database (ae_rules table) with full lifecycle management. Lead Analysts create or edit rules through a web interface; changes enter a DRAFT state.
- The Lead Analyst submits the change, which enters a review queue with a field-level before/after diff visible to the Approver. The Approver reviews the diff, adds comments, approves or rejects, and approved changes can be promoted to the PROD environment. Every action is time-stamped and attributed in the Version History Log.

Three-Popup Natural Language Rule Builder

Instead of writing SpEL expressions or raw code, analysts use a guided three-step popup system to build rule conditions:

- Step 1: Selects a variable from the typed variable registry (tu.vantageScore, de.creditGrade, contact. State, etc.);
- Step 2: Selects the condition operator appropriate for that data type (less than, greater than, between, is one of, etc.);
- Step 3: Enters the value — either a fixed number or a runtime-resolved cutoff reference. The builder compiles a human-readable sentence and the underlying condition expression simultaneously and uses a portal-based popup architecture to avoid DOM clipping issues.

Cutoff Tracker with Production Visibility

- Cutoffs are stored both as a JSON snapshot (cutoff_group_snapshot) and as individual rows (embedded_cutoff_entry) with one database row per dimension key (creditGrade x channel x state combination).
- This enables the Cutoff Tracker UI to show a queryable production table: what is the cvScoreCutoff for grade A1, channel CKPQ, state TX right now? Changes are tracked row-by-row with previous_value and is_changed flags.

Flyway-Style Version Manager

- Every save to the rule engine requires a semantic version string (x.y.z). The system rejects saves if the version already exists for the same scope — a collision detection mechanism analogous to Flyway's V1__description.sql numbering.
- Multiple analysts can work in parallel on different versions without conflict. The version lifecycle follows: DRAFT → PENDING_REVIEW → APPROVED → PROMOTED → optionally ROLLED_BACK. Each version stores a full config snapshot for point-in-time restoration.

AI-Powered UW Excel Ingestion

This is the flagship AI feature. When the UW Analytics team uploads their annotated Excel specification (AE_Sample_Spec.xlsx), the ingestion service:

  - Parses the Rules sheet and detects all yellow-highlighted cells (FFFFFF00 fill code) indicating UW team changes.
  - Cross-references each highlighted rule against the ae_rules config table — surfacing any rules present in the UW spec but missing from the system.
  - Loads the active rule engine bundle from the JVM cache and database.
  - Runs an AI suggestion engine that compares the Excel specification against the current engine configuration and generates typed change suggestions: CUTOFF_CHANGE, OPERATOR_CHANGE, FIELD_RENAME, ADD_RULE, REMOVE_RULE, TAG_MODIFY, CHANNEL_EXTENSION, CONDITION_REWRITE, SEGMENT_CHANGE.
  - Each suggestion includes a before/after diff and a generated JSON payload that applies the change immediately to the engine.

Embedded Rule Engine with Variable Registry

The platform includes a full embedded rule execution engine with a 30-variable registry (tu.*, ccr.*, de.*, clarity.*, contact.*, app.*), multi-dimensional cutoff resolution, provider-based filtering (rules are skipped if a required data provider — tu, ccr, clarity — is unavailable for a given request), and batchId-based JVM cache invalidation. Rules saved to the embedded engine are automatically mirrored to the ae_rules config table, maintaining alignment between both systems.

Where AI Is Used

UW Excel Yellow-Cell Detection Apache POI reads the annotated Excel file and precisely identifies which cells carry a FFFFFF00 (yellow) fill, meaning the UW team intends a change. This is deterministic pattern recognition on the raw file format.	**Config Cross-Reference & Gap Analysis** The system cross-references every highlighted rule ID against the ae_rules database and the active rule engine bundle, producing a gap report: which rules are in the UW spec but not in the system, and vice versa.
AI Change Suggestion Engine For each yellow-highlighted row, the engine compares the Excel specification against the live config and generates typed, actionable change suggestions — not just "something changed" but specifically what type of change (cutoff value, operator, field name, new rule, disabled rule, tag, channel override) with exact before/after values.	**Natural Language → Condition Compiler** The three-popup builder compiles human-readable rule sentences ("tu.vantageScore is less than 500") into executable condition expressions, making rule authoring accessible to non-engineers.
SpEL Auto-Translation When a rule description is provided but no condition expression is written, the SpEL translator attempts to auto-generate the expression from 17 regex-pattern rules covering all standard AE rule types.	**Version Collision Intelligence** The Flyway-style version manager detects collisions, identifies the existing version's owner and status, and suggests the next available patch version automatically.

Why AI Is Needed for This Use Case

- The AE Rule Engine addresses ADF's Affiliate Engine configuration management problem with a five-layer architecture.
- The Config Management layer provides a database-backed single source of truth for all AE rules, state constraints, and channel configurations, with a governed DRAFT → PENDING_REVIEW → APPROVED → PROMOTED workflow.

- Lead Analysts author changes; Approvers review field-level diffs; every action is logged to the Version History with timestamps and attribution.
- The Embedded Rule Engine layer executes rules in real time, resolving multi-dimensional cutoffs (creditGrade × channel × state) through an indexed variable registry of 30 typed variables, with batchId-based JVM cache invalidation ensuring sub-millisecond rule evaluation latency.
- The Natural Language Rule Builder eliminates the need for engineers to hand-write condition expressions: a three-step guided popup system (variable → operator → value) compiles human-readable sentences into executable conditions, using a portal-based popup architecture that avoids DOM clipping in complex layouts.
- The Flyway-Style Version Manager enforces unique semantic versioning (x.y.z) per config scope, with collision detection that prevents two analysts from overwriting each other's changes — the precise problem that Flyway solves for database migrations, now applied to lending rule configuration.
- The AI-Powered UW Excel Ingestion module is the centrepiece innovation: it reads the UW team's yellow-annotated Excel specification, cross-references every highlighted rule against both the ae_rules config table and the live engine bundle, and produces a typed change suggestion report (CUTOFF_CHANGE, OPERATOR_CHANGE, FIELD_RENAME, ADD_RULE, REMOVE_RULE, TAG_MODIFY, CHANNEL_EXTENSION) with before/after diffs and one-click application payloads. Analysing the actual ADF Excel spec (AE_Sample_Spec.xlsx) surfaced five real change clusters across AE_TUSOFT_CV_SCORE (sub-rules c1–c5, cutoffs 500/585/576/570 across channels), AE_THIN_FILE_RULE (sub-rules g, h, i1 for CKPQ, LT, ML channels), AE_RISK_RULE (i2, cutoffs 6 and 50), AE_V11_CCR and AE_V11_TU_FT_CCR (model score cutoff 505).
- The system surfaces these automatically, with no manual interpretation required. This solution directly eliminates ADF's highest-risk manual process, introduces regulatory-grade auditability, and enables the UW and engineering teams to collaborate on a shared system rather than a shared spreadsheet.

4. Final Output / Demo

Type of Output Built

Primary Type	Full-Stack Web Application (Prototype → Production-Grade)
Frontend	React 18 + Vite, served on http://localhost:5173
Backend	Spring Boot 3.2 + Java 17 REST API on http://localhost:8080
Database	H2 (dev) / PostgreSQL (prod) — 12 JPA entities, auto-schema
AI / ML	Apache POI colour detection, rule diff engine, SpEL translator
Version Control	Flyway-style semantic versioning with collision detection

What the Final Output Demonstrates

- End-to-end Affiliate Engine rule lifecycle: create → review → approve → promote → audit
- AI ingestion of the actual ADF Excel spec (AE_Sample_Spec.xlsx) with 11 yellow-highlighted rows detected
- Cross-check: which highlighted rules are present in the ae_rules Config tab vs missing
- 9 typed AI change suggestions generated automatically from the Excel diff
- Three-popup natural language rule builder with live condition preview, working portal popups
- Cutoff Tracker showing production rows per creditGrade × channel × state dimension
- Flyway-style version collision detection: saving v1.2.0 twice for the same scope → rejected with suggested v1.2.1
- Field-level before/after diff in the Review Queue (Approver view)
- Version History Log with timeline, filters by entity type and action

Technology Stack — Full Detail

Layer	Technology	Purpose
Frontend	**React 18 + Vite**	Component-based SPA with hot module replacement
Frontend	**React Router v6**	Client-side routing for 14 pages
Frontend	**Axios**	HTTP client for REST API communication
Frontend	**Lucide React**	Icon library — consistent iconography
Frontend	**React Hot Toast**	Toast notification system
Frontend	**date-fns**	Date formatting for audit trail timestamps
Backend	**Spring Boot 3.2**	REST API framework with auto-configuration
Backend	**Spring Data JPA**	12 JPA entities, H2/PostgreSQL persistence
Backend	**Spring Security**	Role-based access (Lead Analyst, Approver)
Backend	**Apache POI 5.2.5**	Excel file parsing — cell colour detection

Layer	Technology	Purpose
Backend	Jackson	JSON serialisation of rule bundles and cutoffs
Database	H2 (dev)	In-memory, auto-schema for fast development
Database	PostgreSQL (prod)	Production persistence with full ACID compliance
AI / ML	Cell Colour Detection	FFFFFF00 (yellow) fill detection via POI XSSFColor
AI / ML	Change Suggestion Engine	Rule-based inference: 9 typed change categories
AI / ML	SpEL Translator	17 regex patterns → Spring Expression Language
AI / ML	Version Collision Detect.	Flyway-style unique-constraint on version + scope

End-User Journey / Demo Flow

Journey 1 — Lead Analyst creates a rule change

1	Developer	Opens Rules page — sees all 20+ AE rules with colour-coded phase badges and highlighted Rule IDs
2	Developer	Clicks New Rule or edits AE_TUSOFT_CV_SCORE — opens the guided three-popup Rule Builder
3	Developer	Step ① selects variable tu.cvScore from the typed registry; Step ② selects "is less than" operator; Step ③ enters value 585 — condition preview shows "tu.cvScore is less than 585" in green
4	Developer	Sets version 1.2.0 — system validates no collision. Saves — rule is mirrored to ae_rules table and embedded engine.
5	Developer	Submits for review — rule enters PENDING_REVIEW state in the Review Queue
6	TL/Manager	Opens Review Queue — sees field-level diff: Cutoff 500 (red, strikethrough) → 585 (green, bold)
7	TL/Manager	Approves with comment "Aligned with UW spec update Q1 2026" — rule moves to APPROVED
8	TL/Manager	Promotes to Production — ae_rules record duplicated to PROD environment, Version History Log updated

Journey 2 — UW Excel ingestion (AI-powered)

1	BA Team	Opens UW Excel → Config page — drags AE_Sample_Spec.xlsx onto the drop zone
2	System (AI)	POI parser scans 92 rows of the Rules sheet — detects 11 yellow-highlighted rows across 5 rules
3	System (AI)	Config Cross-Check tab shows which rule IDs are in the ae_rules table vs missing

4	System (AI)	Suggestion engine generates 9 typed change suggestions: 4× CUTOFF_CHANGE, 2× CHANNEL_EXTENSION, 1× ADD_RULE, 1× TAG_MODIFY, 1× OPERATOR_CHANGE
5	TL/Manager	Reviews each suggestion with before/after diff — sees AE_TUSOFT_CV_SCORE cutoff change: 500 → 585
6	TL/Manager	Clicks "Apply" — change is written to embedded engine and mirrored to ae_rules table
7	TL/Manager	Clicks "Apply All HIGH" — all 4 high-priority changes applied in one batch

Sample Input

UW Excel Specification — AE_Sample_Spec.xlsx (actual ADF file)

Yellow cells detected: • Row 33–37: AE_TUSOFT_CV_SCORE (c1–c5) — cutoffs 500 / 585 / 576 / 570 across All / MO / ML / E1_E2 segments • Row 46–48: AE_THIN_FILE_RULE (g, h, i1) — channel-specific cutoffs for CKPQ, LT, ML • Row 49: AE_RISK_RULE (i2) — Cutoff_1: 6, Cutoff_2: 50 for CKPQ, LT, ML, MO, QS • Row 70: AE_V11_CCR — model score cutoff 505 • Row 71: AE_V11_TU_FT_CCR — model score cutoff 505

Sample Output

AI Change Suggestion — CUTOFF_CHANGE (HIGH severity)

Rule: AE_TUSOFT_CV_SCORE | Sub-rule: c1 | Section: TU Pull Title: Cutoff value change: 500 → 585 Description: UW team has updated the cutoff for rule AE_TUSOFT_CV_SCORE from 500 to 585 in the Excel spec (highlighted yellow). Current Engine Value: 500 Excel Spec Value: 585 Excel Segment: MO and Segment_flag = Others Generated Payload: { "action": "UPDATE_CONDITION", "ruleId": "AE_TUSOFT_CV_SCORE", "oldCondition": "tu.cvScore < 500", "newCondition": "tu.cvScore < 585", "cutoffChange": { "from": "500", "to": "585" } }

Screenshots Attached	See PPTX reference deck — AE_Rule_Engine_HTML_to_PPT.pptx
Demo Video Link	
Live Demo Link	http://localhost:5173 (run locally per startup guide)
Repository / Archive	GitHub - varalakshmigurram/AE-ReImagined-Fintelingence-AI: \AE Rule Engine - Fintellingence AI · GitHub
Startup Guide	AE_RuleEngine_Startup_Guide.docx — included in submission

Startup Instructions

Prerequisites: Java 17+, Maven, Node.js 18+

- Terminal 1 (Backend): cd backend && ./mvnw spring-boot:run → http://localhost:8080

- Terminal 2 (Frontend): cd frontend && npm install && npm run dev → http://localhost:5173
- H2 Console: http://localhost:8080/h2-console (JDBC URL: jdbc:h2:mem:ruleengine)
- Data seeded automatically on startup: 20 AE rules, 35 states, 7 channels, 30 variables

5. Business Impact

Quantified Efficiency Gains

Layer	Technology	Purpose
Rule Deployment Speed	**Manual: 2–4 hours per change**	AE Rule Engine: 15 minutes end-to-end (author → approve → deploy)
Error Rate	**Transcription errors in ~15% of changes**	Zero transcription: system reads Excel directly
Audit Readiness	**Reconstruct from emails + git (hours)**	Instant: Version History Log with full timeline
UW Spec Alignment Check	**Manual review: 30–60 minutes**	AI cross-check: seconds
Configuration Rollback	**Not possible without code revert**	One-click: restore any previous PROMOTED version
Parallel Analyst Work	**Blocked — single Excel file**	Parallel: Flyway-style version isolation

Risk Reduction

- Eliminates the single highest-risk manual step in AE operations: Excel-to-engine transcription
- Provides regulatory-grade audit trail satisfying change management requirements for credit decision systems
- Prevents environment drift: TEST and PROD configs tracked separately with explicit promotion gate
- Cutoff collision detection ensures no two changes silently overwrite each other

Strategic Value

- Scalable: as ADF adds affiliate partners or new rule categories, the system absorbs them without process changes
- Self-documenting: every rule has a description, version history, and the UW spec context that created it
- Platform for future AI: the variable registry, version history, and structured rule format are the foundation for predictive rule impact analysis and automated conflict detection
- Reusable pattern: the same governance architecture applies to any ADF configuration system beyond the Affiliate Engine

"From spreadsheet chaos to governed, AI-assisted, auditable rule management."